

Advanced Data Engineering Notes

| Day 37: Databricks Advanced

Day 37 transitions from basic PySpark and Delta Lake operations to managing production-ready workloads and optimizing them at scale.

1. Jobs & Workflows

- **Task Orchestration:** Define multiple tasks within a job. Task B can be set to run only if Task A succeeds (e.g., Extract → Transform → Load).
- **Job Clusters vs. All-Purpose Clusters:** Always use Job Clusters for scheduled workflows. They spin up, execute the job, and terminate, providing significant cost savings compared to leaving an All-Purpose cluster running.

2. Databricks SQL Analytics

Databricks SQL provides a traditional Data Warehouse experience directly on top of your Data Lake (Lakehouse architecture).

- **SQL Warehouses:** Specially tuned compute clusters optimized specifically for SQL execution rather than general PySpark jobs.

3. Performance Tuning

Optimizing workloads is critical for minimizing cloud costs and meeting SLAs.

- **Adaptive Query Execution (AQE):** Spark dynamically re-optimizes query plans during execution based on runtime statistics.
- **Z-Ordering:** Co-locates related information in the same set of files in Delta Lake. Great for speeding up queries that frequently filter on high-cardinality columns.
- **Broadcast Joins:** When joining a massive Fact table with a tiny Dimension table, broadcast the tiny table to all worker nodes to avoid network shuffling.

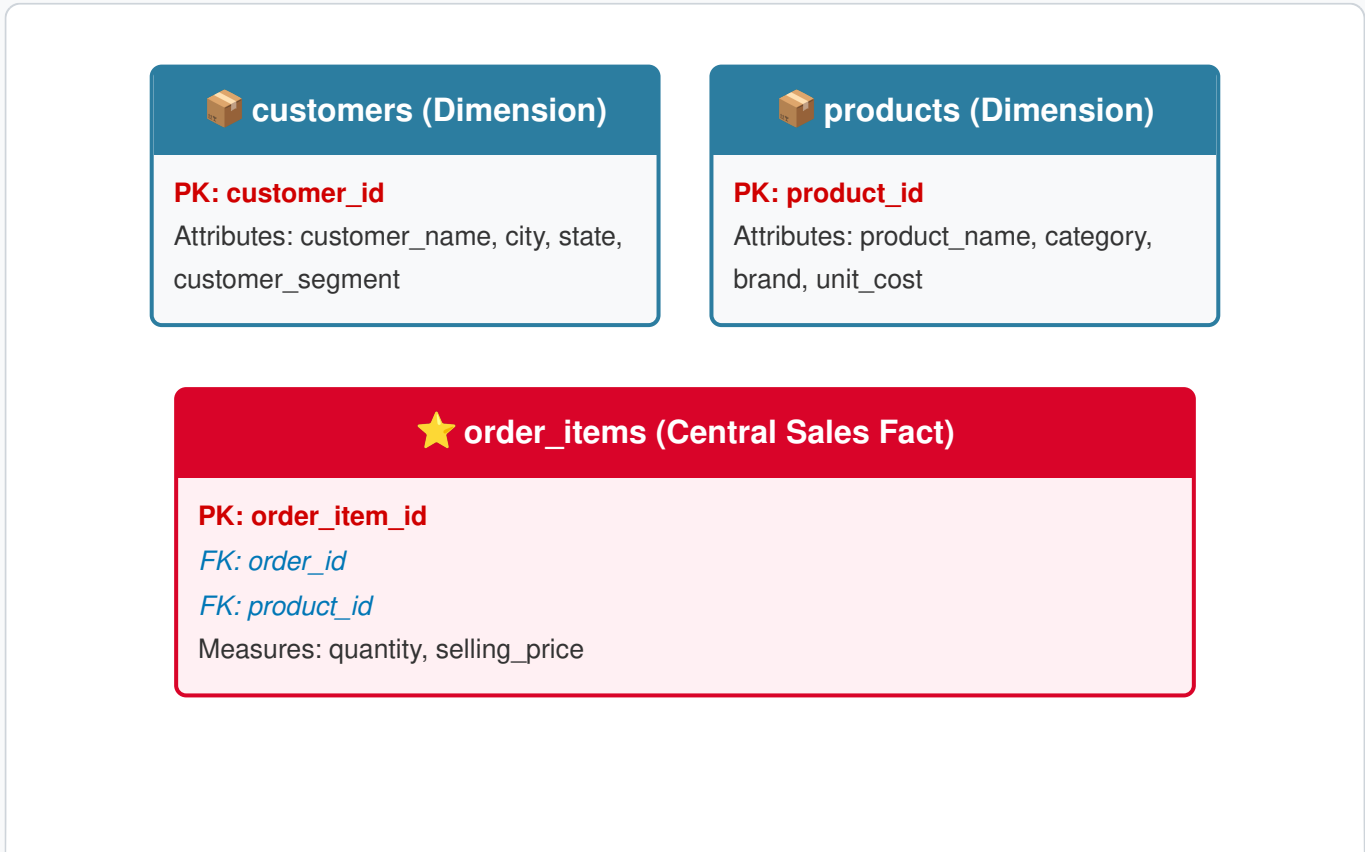
Data Modeling: Facts & Dimensions

Data modeling structures your raw data into a format optimized for business intelligence. The industry standard is Dimensional Modeling (the Star Schema).

Concept	Description
Fact Tables	Store quantitative metrics (measurements) of a business process (e.g., Sales Amount, Quantity). They grow rapidly and consist primarily of Foreign Keys and numeric measures.
Dimension Tables	Provide the "who, what, where, when, why" context. They contain descriptive attributes (e.g., Customer Name, Product Category).
Star Schema	An architecture where a central Fact table is surrounded by Dimension tables, minimizing complex joins and maximizing BI performance.

Dataset Star Schema Diagram

Based on your specific e-commerce dataset files, here is the logical flow mapping the relationships:



orders (Header/Bridge)

PK: order_id

FK: customer_id

Attributes: order_date, payment_mode,
order_status

returns (Secondary Fact)

PK: return_id

FK: order_id

Attributes: return_date, return_reason

Day 41: Power BI Fundamentals

Power BI is an interactive data visualization software with a primary focus on business intelligence.

1. Data Connectivity

- **Import Mode:** Data is loaded entirely into Power BI's internal memory. Offers the fastest performance and full DAX support. Best for datasets under 1GB.
- **DirectQuery:** Power BI leaves the data in the source database and sends SQL queries on the fly. Best for massive datasets or near-real-time requirements.

2. DAX Basics (Data Analysis Expressions)

- **Calculated Columns:** Evaluated row-by-row during data refresh. Takes up memory space.
- **Measures:** Evaluated on-the-fly based on the context of the visualization (filters). Highly efficient.

Power BI Implementation Step-by-Step Flowcharts

Actionable workflow instructions for transforming your raw CSV dataset into high-impact Power BI visuals.

Phase 1: Data Import & Modeling Flow

Step 1: Get Data → Select Text/CSV → Load all 5 files (customers, products, orders, order_items, returns).

Step 2: Go to the Model View (the relationship icon on the left sidebar).

Step 3: Drag customer_id from customers to orders (Creates a 1-to-Many relationship).

Step 4: Drag order_id from orders to order_items (Creates a 1-to-Many relationship).

Step 5: Drag product_id from products to order_items (Creates a 1-to-Many relationship).

Step 6: Drag `order_id` from orders to returns (Creates a 1-to-1 or 1-to-Many relationship).

Step 7: Go to the Data View, right-click `order_items`, and select **New Measure**.

Step 8: Type: `Total Revenue = SUMX(order_items, order_items[quantity] * order_items[selling_price])`

Phase 2: Building Visualizations (Report View)

Chart A: Total Revenue by Product Category

1. Select **Clustered Bar Chart** from the Visualizations pane.
2. Drag `category` from products to the **Y-axis**.
3. Drag your new `Total Revenue` measure to the **X-axis**.
4. Formatting: Enable "Data Labels" and sort the axis by Total Revenue (Descending).

Chart B: Sales Trend by VIP Customers

1. Select **Line Chart** from the Visualizations pane.
2. Drag `order_date` from orders to the **X-axis** (Keep Year and Month).
3. Drag `Total Revenue` to the **Y-axis**.
4. Drag `customer_segment` from customers to the **Legend**.
5. Filtering: Add a **Slicer** visual to the page, drag `customer_segment` into it, and click "VIP" to filter the page.

Chart C: Return Rate Analysis

1. Select **Donut Chart** from the Visualizations pane.
2. Drag `return_reason` from `returns` to the **Legend**.
3. Drag `return_id` from `returns` to the **Values** field (Ensure the aggregation is set to "Count").
4. Formatting: Go to Detail Labels and change the style to show "Category, Percent of Total".